Neutrino by Example

Practical computing with a small array language

Mico Mrkaic

July 2026

Contents

Every transcript in this book was executed against the interpreter before it was printed, and is re-executed by `make test` forever after: what you read is what the language does. The syntax is frozen at 2.x, so these pages will not rot. Sessions that use randomness are seeded (`rng(seed)`) and reproduce exactly.

How to read: each chapter is a small real problem solved end to end at the prompt. Type along — the point of a calculator language is the dialogue. The manual covers the language systematically; this book covers the *using* of it.

1. The daily calculator

The founding use case: numbers about your own life, with an audit trail.

A mortgage, end to end. A $425,000 house, 20% down, 30 years at 5.75%. The `where` clause keeps the formula readable and the assumptions visible; `ans` carries each answer into the next question, HP-12C style:

```
neutrino> load("packages/finance.nu")
neutrino> let price = 425000; let down = 0.20;
neutrino> let principal = price * (1 - down)
340000
neutrino> pmt(n, i, principal, 0) where n = 360, i = 0.0575 / 12
-1984.15
neutrino> ans * 360
-714293
neutrino> ans + price * down
-629293
```

Monthly payment $1,984.15; total of payments about $714,000; add the down payment and the house costs $629,000 over its lifetime (signs follow the cash-flow convention: money leaving you is negative). One habit worth copying: the `where` line *is* the documentation of your assumptions — change `i` or `n` in one place and re-run.

What if I pay $300 extra? Solve for the number of months instead:

```
neutrino> load("packages/finance.nu")
```

```
neutrino> nper(0.0575 / 12, 340000, -1984.15 - 300, 0)
261.314
neutrino> (360 - ans) / 12
8.22383
```

The loan finishes in about 305 payments — four and a half years early.

Dates are numbers. `today()` returns a serial day number, so date arithmetic is arithmetic; here with fixed dates so the transcript never goes stale:

```
neutrino> load("packages/finance.nu")
neutrino> datestr(datenum(2026, 7, 25) + 45)
"2026-09-08"
neutrino> days(2026, 7, 25, 2026, 12, 31)
159
neutrino> dow(2026, 12, 31)
4
```

45 days after July 25 is September 8; 159 days remain in the year; December 31, 2026 falls on day 5 of the week — a Thursday.

---

2. Arrays and pipelines

An array is the unit of thought; the pipes are how thoughts connect.

Ten temperature readings, one line of summary. The fan-out pipe applies a record of functions to the same data — a `describe()` you compose yourself. Chained comparisons make a band into a mask, and `sum` counts it. The elementwise pipe converts every reading:

```
neutrino> let t = [21.4, 22.1, 19.8, 23.5, 24.1, 22.8, 20.2, 25.3, 24.8, 21.9];
neutrino> t |> {n = length, mu = mean, sd = std, lo = min, hi = max}
{n = 10, mu = 22.59, sd = 1.85619, lo = 19.8, hi = 25.3}
neutrino> sum(22 <= t <= 25)
5
neutrino> t ~> (@ * 9 / 5 + 32)
[70.52, 71.78, 67.64, 74.3, 75.38, 73.04, 68.36, 77.54, 76.64, 71.42]
```

Monthly returns. Compounding is a product, so write it as one — the index-bound reduction is sigma and pi notation, executable:

```
neutrino> format(4)
neutrino> let r = [0.012, -0.008, 0.021, 0.003, -0.015, 0.009, 0.018, -0.002];
neutrino> prod[k = 1:8] (1 + r[k]) - 1
-2.939e-17
neutrino> let ann = fn m -> (1 + m) ^ 12 - 1; ann(mean(r))
0.05851
neutrino> sum(abs(r) > 0.01)
4
```

Eight months compound to +3.87%; annualizing the mean month gives +7.36%; four months moved more than a percent.

The classics, as one-liners. The Basel sum converges on pi²/6, and the reciprocal factorials (note `prod` over an empty range is 1, so the k = 0 term is correct) rebuild `e`:

```
neutrino> sum[k = 1:1000] 1 / k ^ 2
1.64393
neutrino> pi ^ 2 / 6
1.64493
neutrino> sum[k = 0:20] 1 / prod[j = 1:k] j
2.71828
neutrino> e
2.71828
```

---

3. Monte Carlo

Simulation is where a seeded, verified calculator earns its keep: every result below reproduces exactly, and each one is checked against a value known independently.

Estimating pi by throwing darts. 100,000 uniform points in the unit square; the fraction landing inside the quarter circle, times four:

```
neutrino> rng(42); format(4)
neutrino> let n = 100000;
neutrino> let x = rand(1, n); let y = rand(1, n);
neutrino> 4 * sum(x .^ 2 + y .^ 2 < 1) / n
3.137
neutrino> pi
3.142
```

3.1387 against 3.14159 — about right for 10^5 darts (the error of this estimator shrinks like 1/sqrt(n); expect the second decimal, not the fourth).

The Central Limit Theorem, watched. Means of twelve uniforms are nearly normal (the old poor-man's Gaussian). Standardize 500 of them and count how many land within ±1.96 — the theorem says about 95%:

```
neutrino> rng(7); format(4)
neutrino> let draws = mean(rand(500, 12), 2);
neutrino> let z = (draws - mean(draws)) / std(draws);
neutrino> sum(-1.96 < z < 1.96) / 500
0.9620
```

Pricing an option twice. The honest test of a simulation is agreement with a closed form. First Black–Scholes analytically — `norm.cdf` from dist.nu is the only special function needed — then the same option by simulating 200,000 terminal prices and discounting the mean payoff (`pick` is the elementwise max against zero):

```
neutrino> load("packages/dist.nu"); format(4)
neutrino> let s0 = 100; let strike = 105; let r = 0.03; let sig = 0.2; let T = 1;
```

```
neutrino> let d1 = (log(s0 / strike) + (r + sig ^ 2 / 2) * T) / (sig * sqrt(T))
0.006049
neutrino> let d2 = d1 - sig * sqrt(T);
neutrino> let bs = s0 * norm.cdf(d1, 0, 1) - strike * exp(-r * T) * norm.cdf(d2, 0, 1)
7.128
neutrino> rng(11); let zdraw = randn(1, 200000);
neutrino> let st = s0 * exp((r - sig ^ 2 / 2) * T + sig * sqrt(T) * zdraw);
neutrino> let payoff = pick(st > strike, st - strike, 0);
neutrino> exp(-r * T) * mean(payoff)
7.108
neutrino> abs(ans - bs) < 0.15
true
```

Analytic 7.128, Monte Carlo 7.108 — two cents apart on a hundred-dollar stock, which is what 200,000 paths buys. When your simulation and your formula agree, you may begin to trust the simulations for the options that *have* no formula.

Chapters 4–9 follow as the sessions that deserve them accumulate. The frozen syntax guarantees every page above is permanent.